Divide and Conquer

1-Number of Zeros in a Given Array

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Answer: (penalty regime: 0 %)

```c
#include<stdio.h>
int count(int a[],int low,int high){

    if (low > high)
        return 0;

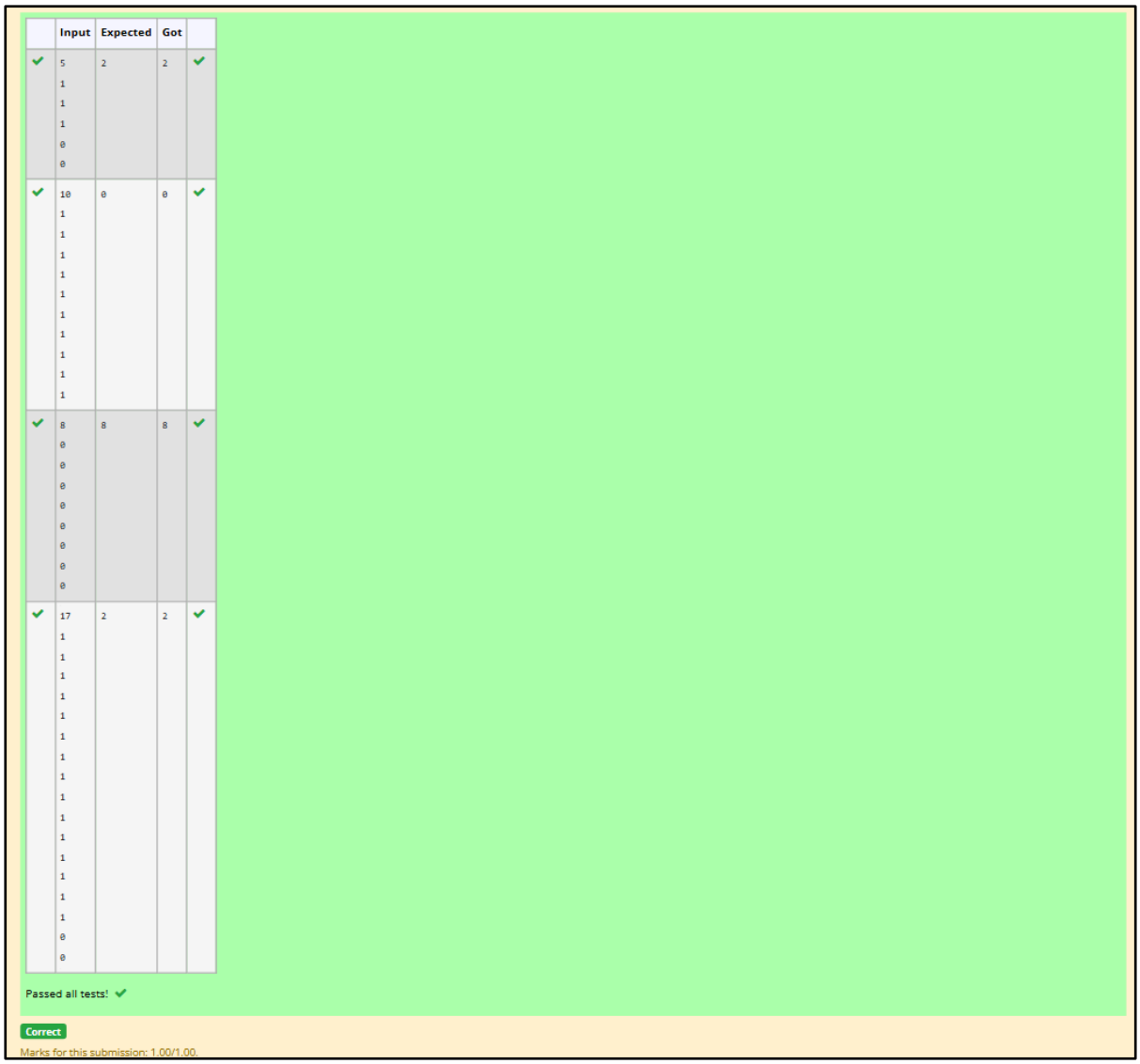
    if (a[low] == 0)
        return high - low + 1;

    int mid = (low + high) / 2;

    if (a[mid] == 1)
        return count(a, mid + 1, high);

    return count(a, low, mid);
}
int main(){
    int m;
    scanf("%d",&m);
    int a[m];
    for(int i=0;i<m;i++){
        scanf("%d",&a[i]);
    }
    printf("%d",count(a,0,m-1));
    return 0;
}
```

Edit with WPS Office



	Input	Expected	Got	
✔	5 1 1 1 0 0	2	2	✔
✔	10 1 1 1 1 1 1 1 1 1 1	0	0	✔
✔	8 0 0 0 0 0 0 0 0	8	8	✔
✔	17 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0	2	2	✔

2-Majority Element

Given an array nums of size n, return *the majority element*.

The majority element is the element that appears more than ⌊n / 2⌋ times. You may assume that the majority element always exists in the array.

Example 1:

Input: nums = [3,2,3]

Output: 3

Example 2:

Edit with WPS Office

Input: nums = [2,2,1,1,1,2,2]

Output: 2

Constraints:

- n == nums.length
- $1 <= n <= 5 * 10^4$
- $-2^{31} <= nums[i] <= 2^{31} - 1$

Answer: (penalty regime: 0 %)

```c
#include<stdio.h>
int maj(int a[],int low,int high){
    if(low==high)
      return a[low];
    int mid=(high+low)/2;
    int left=maj(a,low,mid);
    int right=maj(a,mid+1,high);
    if(left==right)
      return left;
    int l=0;
    int r=0;
    for(int i=low;i<=high;i++){
        if(a[i]==left){
            l++;
        }
        if(a[i]==right){
            r++;
        }
    }
        if(l>r)
            return left;
        else
            return right;
}
    int main(){
    int n;
    scanf("%d",&n);
    int a[n];
    for(int i=0;i<n;i++)
        scanf("%d",&a[i]);
    printf("%d",maj(a,0,n-1));
    return 0;
}
```

	Input	Expected	Got	
✔	3 3 2 3	3	3	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Edit with WPS Office

3-Finding Floor Value

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer: (penalty regime: 0 %)

```c
#include<stdio.h>
int main(){
    int n;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    int x;
    scanf("%d",&x);
    int l=0;
    int h=n-1;
    int f=-1;
    while(l<=h){
        int m=l+(h-l)/2;
        if(arr[m]==x){
            f=arr[m];
            break;
        }
        else if(arr[m]<x){
            f=arr[m];
            l=m+1;
        }
        else{
            h=m-1;
        }
    }
    printf("%d\n",f);
    return 0;
}
```

	Input	Expected	Got	
✔	6 1 2 8 10 12 19 5	2	2	✔
✔	5 10 22 85 108 129 100	85	85	✔
✔	7 3 5 7 9 11 13 15 10	9	9	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Edit with WPS Office

4-Two Elements sum to x

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

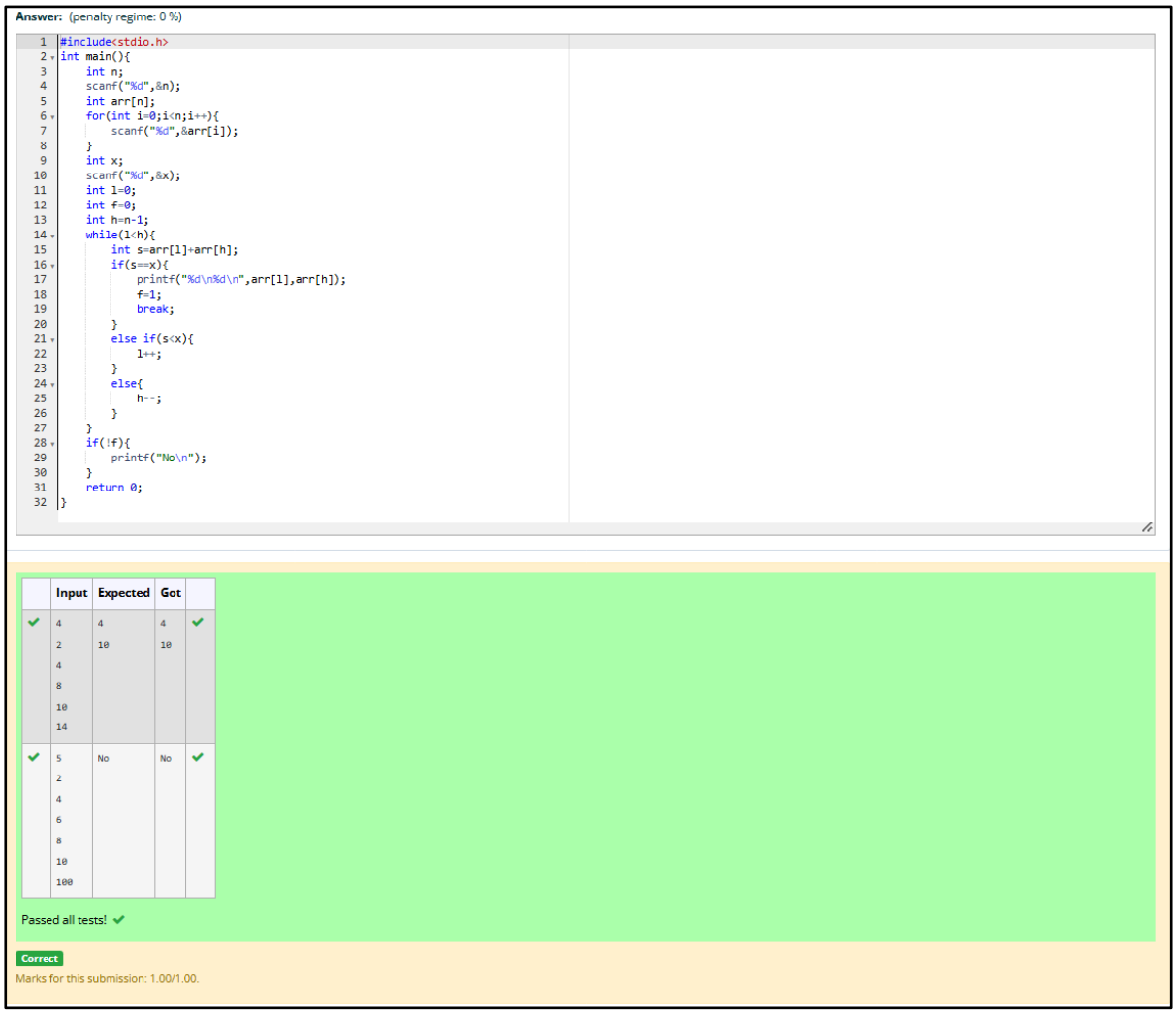
Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")



Answer: (penalty regime: 0 %)

```c
#include<stdio.h>
int main(){
    int n;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    int x;
    scanf("%d",&x);
    int l=0;
    int f=0;
    int h=n-1;
    while(l<h){
        int s=arr[l]+arr[h];
        if(s==x){
            printf("%d\n%d\n",arr[l],arr[h]);
            f=1;
            break;
        }
        else if(s<x){
            l++;
        }
        else{
            h--;
        }
    }
    if(!f){
        printf("No\n");
    }
    return 0;
}
```

	Input	Expected	Got	
✔	4 2 4 8 10 14	4 10	4 10	✔
✔	5 2 4 6 8 10 100	No	No	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

5-Implementation of Quick Sort

Write a Program to Implement the Quick Sort Algorithm

Edit with WPS Office

Input Format:
The first line contains the no of elements in the list-n
The next n lines contain the elements.

Output:
Sorted list of elements

Answer:

```c
#include<stdio.h>
int main(){
    int n;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    for(int i=0;i<n-1;i++){
        for(int j=i+1;j<n;j++){
            if(arr[i]>arr[j]){
                int temp=arr[i];
                arr[i]=arr[j];
                arr[j]=temp;
            }
        }
    }
    for(int i=0;i<n;i++){
        printf("%d",arr[i]);
        if(i<n-1){
            printf(" ");
        }
    }
    printf("\n");
    return 0;
}
```

	Input	Expected	Got	
✔	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✔
✔	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✔
✔	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Edit with WPS Office